

Problem A. White Night 2

Start with the special case $k = 1$. Choose the positions of the 1's that are deleted by operation type one. No two chosen positions may be adjacent. A chosen 1 near the boundary contributes length 2, while a chosen internal 1 contributes length 3. Thus the task is to minimize the number of chosen 1's whose total contribution covers the whole string; this can be done greedily or by a small DP.

For general k , a tempting but incomplete rule is to choose 1's with mutual distance at least k and use the analogous contribution formula. This fails for patterns where the last deleted 1 interacts with both sides. Instead, enumerate the last 1 deleted. After this deletion, the left and right parts become independent subproblems: the first k positions of each side cannot be selected, and the only requirement is that the remaining length becomes at most k .

A direct DP over the last selected position gives $O(n^2)$ time. It can be improved by splitting into two cases. If a selected 1 is near the boundary, enumerate the first selected position and the number of selected positions is $O(n/k)$. Otherwise, erase boundary 1's and check whether enough pairwise-separated 1's exist using two pointers. A final one-dimensional DP handles the lower-bound case. This gives an $O(n)$ or $O(n \log n)$ implementation depending on data-structure choices.

Problem B. Alternative Accounts 2

No analysis was provided in the source editorial at the time used.

Problem C. Ring of Beads

No analysis was provided in the source editorial at the time used.

Problem D. DFS Order 6

No analysis was provided in the source editorial at the time used.

Problem E. Back Edges

No analysis was provided in the source editorial at the time used.

Problem F. Sticks 2

No analysis was provided in the source editorial at the time used.

Problem G. Art of Data Structures

No analysis was provided in the source editorial at the time used.

Problem H. Guess the Permutation

If the position of the value 1 is known, the reconstruction is simple. Pick an arbitrary y and query (x, y) . The returned number tells us where to continue; repeating this process follows a cycle. Once a cycle closes, start again from an unresolved position. Eventually all values are determined.

If the position of 1 is not known, temporarily pretend that a candidate position contains 1. Whenever a smaller value is discovered, switch the candidate to that position and continue. The progress is similar to the Euclidean algorithm by subtraction: every two extra operations reduce the true candidate value by at least a factor of two. Therefore at most $n + 2 \log n$ queries are needed.

After the query graph is built, the permutation values can be recovered by a topological-sort-like reconstruction over the discovered relations.

Problem I. Defense Line

No analysis was provided in the source editorial at the time used.

Problem J. Emergency Pipeline Repair

The geometric core is the shortest line segment that intersects all given lines, also known as a shortest transversal problem.

Fix the direction of the segment by rotating the plane so that the segment is vertical. Then the problem becomes minimizing the vertical distance between the upper envelope and the lower envelope of the transformed lines. If we relax the requirement to connecting a point on the upper envelope to a point on the lower envelope, the optimum can be found with a two-pointer walk because both envelopes are convex.

As the segment direction rotates, the order changes only at line angles, so there are $O(n)$ orientation intervals. The upper and lower envelopes that matter are boundaries of unbounded cells in the line arrangement. By the zone theorem, their total combinatorial size over all intervals is linear. They can be built by half-plane-intersection sweeps from $y = -\infty$ and $y = +\infty$.

Combining these ingredients gives an $O(n \log n)$ algorithm.

Problem K. Call You With Your Name 3

If the original string is not a Lyndon string, taking it as one part is already sufficient.

Otherwise, let a be the smallest character. Analyze the runs of a in the string. If all occurrences of a form one prefix run, then no valid nontrivial partition exists. If there are at least two runs of a and the total number of a 's is at least three, cut off the first run together with the following larger block; this creates a first part that is lexicographically smaller than the rest.

The delicate case is when there are exactly two occurrences of the smallest character. Then the first string in the partition has almost fixed form, except for the suffix length. Choose the increasing prefix,

minus one carefully chosen position, so that the first character of the remaining string is maximized, and then recurse on the suffix.

All decisions can be made with the positions and counts of the minimum character, so the implementation is linear.

Problem L. Hanoi Tower Puzzle

No analysis was provided in the source editorial at the time used.

Problem M. Minesweeper

Use the planarity of the graph. First orient the edges so that every vertex has indegree at most 5. This can be done by repeatedly removing a vertex of degree at most 5 and orienting edges when vertices are restored.

Run the process twice. In the second run, restore vertices in the reverse removal order. When a vertex is restored, all but at most five of its neighbors are already known. If the vertex has no mine, the displayed number a_v can be one of only six possible values, depending on how many of the unknown in-neighbors contain mines. Therefore, if we choose a_v outside these six values, the vertex is forced to contain a mine.

The construction assigns numbers so that ordinary mine vertices avoid the six non-mine options. A constant number of exceptional vertices remain; their information is saved separately in a short binary string. This completes the encoding and decoding strategy sketched in the official analysis.